Chapter 11

dplyr Subsetting

by Lorraine Gaudio

Version May 29, 2026

Contents

1 Overview

In this chapter, you will learn how to subset data efficiently with dplyr by combining the pipe `%>%` with two core verbs: `select()` for choosing columns and `filter()` for keeping rows that meet specific conditions. You will also continue working in Quarto, which is the course's reproducible workflow for combining code, results, and written interpretation in one document. Together, these tools help you move from "writing R commands" to building a clear, repeatable analysis process. The chapter closes with applied practice using datasets such as `mtcars`, `gapminder`, and `us_contagious_diseases`.

This chapter matters because subsetting is one of the most common tasks in data analysis. Before you can graph, summarize, model, or answer a research question, you usually need to isolate the relevant cases and variables. In later coursework, you will rely on these skills to prepare datasets for visualization, descriptive statistics, hypothesis testing, and project-based analysis. In professional data work, analysts constantly filter records, choose relevant fields, remove duplicates, and create smaller, targeted datasets for reporting and decision-making. Learning to do this with readable `dplyr` code will make your work faster, easier to check, and easier for other people to follow.

By the end of Chapter 11 you will be able to:

- Use the pipe `%>%` to connect multiple data-wrangling steps into a readable workflow.
- Distinguish between `select()` for choosing columns and `filter()` for keeping rows.
- Apply `select()` to keep, reorder, or exclude variables from a data frame.
- Apply `filter()` to keep rows that meet one or more logical conditions.
- Construct filtering conditions with relational operators such as `>`, `<`, `==`, `!=`, `<=`, and `>=`.
- Combine conditions with `&`, `|`, and `%in%` to create multi-criteria subsets.
- Interpret how Boolean logic affects which observations are kept in a dataset.
- Build pipelines that combine `filter()` and `select()` to create focused, analysis-ready data subsets.
- Produce smaller, relevant datasets that are ready for later visualization, summarization, and modeling.
- Use Quarto to organize code, output, and written interpretation in a reproducible analysis document.

Keep these goals in mind as you move through each section.

2 Quarto

In earlier chapters, you used an R Notebook as a reproducible thinking document: code, output, and memo-style writing in one place. **Quarto** keeps that same basic workflow, but the file is usually a `.qmd` file instead of an `.Rmd` file, and chunk controls are written in YAML-style lines that begin with `#|` inside the code chunk. Quarto is the next-generation version of R Markdown, and RStudio supports creating Quarto documents from `File > New File > Quarto Document…` and rendering them with the Render button.

What are the benefits of Quarto over the R Notebook?

- Quarto supports multiple languages (R, Python, Julia, Observable JavaScript) in the same document. R Notebooks only support R.

To learn more about Quarto, check out the Quarto website. You can also find a helpful Quarto Cheat Sheet.

2.1 R Notebook → Quarto

Quarto will be **similar** to an R Notebook in several ways. You still write normal paragraphs outside code chunks. You still run R code inside chunks. You still use Markdown headings to create an outline. You still render the document to create a readable report.

What changes is the file extension is `.qmd`, the YAML uses `format:` instead of the old `output:` style from R Markdown and the chunk controls are written with `#|` lines at the top of the chunk.

Create a Quarto File

1. For those of you who **have GitHub** and a course project, navigate to your course folder and open your course project. This will open RStudio. Click "Pull" in the Git pane to get the latest version of the course materials. After completing work, save, stage, commit, and push.

If you **don't have GitHub** or a course project, just open RStudio and set your working directory to your course folder.

2. Open a new Quarto document in your course folder and save it as: `chapter11_notes.qmd` (`File > New File > Quarto Document`).

Just like an R Notebook template, a new Quarto file will open with sample text. Start fresh:

- Keep the YAML header at the top.
- Delete the example body text below it.
- Replace that content with your own notes, code, and memos.

2.2 The YAML Header

At the top of the document is the **YAML header**. This is document metadata, not R code. In Quarto, the header is still surrounded by three dashes (`---`) and uses `key: value` pairs. Quarto uses fields such as `title`, `author`, `date`, and `format`.

```yaml
---
title: "Chapter 11 Notes"
subtitle: "dplyr Subsetting"
author: "YOUR NAME"
date: today
format:
  html:
    code-fold: true
execute:
  warning: true
  message: true
---
```

2.2.1 What each part does:

- `title:` gives your document a title.
- `subtitle:` gives your document a subtitle (optional).
- `author:` puts your name on the document.
- `date:` today inserts the current local date automatically. Quarto Dates and Date Formatting has more options for date formatting and dynamic date keywords.
- `format:` html tells Quarto to render HTML output.

- `code-fold:` true lets readers collapse code in the HTML. You can change this to `false` if you want code to always show. Notice that the `true` and `false` values are not capitalized. This is a YAML convention and different from R's `TRUE` and `FALSE`.
- `execute:` sets default chunk behavior for the whole document. In this case, `warning: true` and `message: true` means that if any code chunks produce warnings or messages, they will be included in the output. You can override these defaults in individual chunks with chunk options or you can set them to `false` if you want to hide warnings and messages in your final HTML.

2.3 Markdown Headings

Quarto uses Markdown for normal text, just like your notebook work. Headings are created with `#`, `##`, `###`, and so on. Those headings create the structure of your document.

Use this outline in your notes file:

```
# Overview

# Quarto
```

2.4 Memoing in Quarto

Quarto is not just for tracking your statistical code. It is also where you document your thinking, reasoning, and understanding. That means your memos should be written as normal text outside the chunk, not jammed into code chunk comments unless the note is specifically about the code line itself. This part should feel familiar.

Outside a code chunk write normal sentences and paragraphs. This is where your memoing goes. Inside a code chunk, write R code. If you want to leave yourself *about the code* inside the code chunk, use # so R treats it as a comment.

A Quarto R code chunk still uses fenced backticks with `{r}`. The difference is that chunk options are written as YAML-style lines beginning with `#|` at the top of the chunk. Quarto supports chunk options such as `eval`, `echo`, `output`, `warning`, `error`, and `include`, and those can be set globally in the YAML header or locally in one chunk.

The `#` means different things depending on where you type it. Outside a code chunk, `#` creates a heading. Inside a code chunk, the `#` creates an R comment. At the top of a Quarto chunk, `#|` sets a chunk option.

“‘{r}

#| eval=FALSE

your R code

x <- 1:5

mean(x)

“‘

The “eval=FALSE” chunk option means the code will not run when you render the document. This is useful for code that is just an example or when you want to keep code with an error. You can also use it on code chunks that `install.packages()`.

3 Packages

Speaking of installing packages, you will need to have the `dplyr` and `dslabs` packages installed and loaded to follow along with the code examples in this chapter.

`dplyr` is part of the **tidyverse**, a collection of R packages designed for data science. It provides a set of functions (called "verbs") that make it easy to manipulate and analyze data frames. The main verbs in `dplyr` are `select()`, `filter()`, `mutate()`, `summarize()`, and `arrange()`. In this chapter, we will focus on `select()` and `filter()` for subsetting data.

Recall that we need to download packages **once**. If needed, install the required packages we will use in this lesson. After installation, comment `#` out the `install.packages` code.

```
#  Type this in a Quarto code chunk
# You should already have dslabs from previous chapters, but if not, install it now.
install.packages("dslabs")

install.packages("dplyr")
## Or install the whole tidyverse, which *includes* dplyr and other useful packages:
install.packages("tidyverse")
```

In Quarto, you can set `eval=FALSE` in the chunk option when you want to keep code (like `install.packages()`) without breaking your render.

After installing the package, you need to load it into your R session using the `library()` function. Each time you open R, you will need to "library" the packages. This makes the functions in the package available for use.

```
#  Type this in a Quarto code chunk
library(dslabs)
```

```
## Warning: package 'dslabs' was built under R version 4.5.3
```

```
library(dplyr)
```

```
## Warning: package 'dplyr' was built under R version 4.5.2
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

You can turn off the warning messages by using the Quarto chunk option `warning=FALSE` in the YAML header or in individual chunks (`#| warning=FALSE`). This is useful for suppressing messages from loading packages.

4 The Pipe (%>%)

The `dplyr` package for data manipulation. It will help you with filtering rows, selecting columns, creating variables, summarizing data. It uses a special operator called the **pipe** (`%>%` or `|>`) to chain together multiple operations in a clear and readable way. The pipe chains steps into a readable workflow from left to right. The pipe can be read as "and then" or "goes into". It takes the output of the left-hand side and feeds it as the first argument to the function on the right-hand side.

In tidyverse code, the pipe is `%>%`, so `x %>% f()` means "take `x`, then apply `f()`. You can also choose to use the alternative built-in pipe `|>` from base R.

Goal: Let's see how the pipe works with a simple `head()` example. We will use the `mtcars` dataset.

```r
#  Type this in a Quarto code chunk
mtcars %>% head()
```

```
##                    mpg cyl disp  hp drat    wt  qsec vs am gear carb
## Mazda RX4         21.0   6  160 110 3.90 2.620 16.46  0  1    4    4
## Mazda RX4 Wag     21.0   6  160 110 3.90 2.875 17.02  0  1    4    4
## Datsun 710        22.8   4  108  93 3.85 2.320 18.61  1  1    4    1
## Hornet 4 Drive    21.4   6  258 110 3.08 3.215 19.44  1  0    3    1
## Hornet Sportabout 18.7   8  360 175 3.15 3.440 17.02  0  0    3    2
## Valiant           18.1   6  225 105 2.76 3.460 20.22  1  0    3    1
```

This is equivalent to `head(mtcars)`, but the pipe version reads more like a sequence of actions: "Start with mtcars, and then show the head." The pipe helps avoid nested functions and keeps code in a logical order.

It is common to break lines after the pipe for readability, especially when chaining multiple operations. Each step is on its own line, making it easier to read and understand the flow of data manipulation.

Goal: Use the pipe and break the lines. Show the first six rows of the `mtcars` dataset.

```r
#  Type this in a Quarto code chunk
mtcars %>%        # start with data
  head()          #   shows first six rows
```

```
##                    mpg cyl disp  hp drat    wt  qsec vs am gear carb
## Mazda RX4         21.0   6  160 110 3.90 2.620 16.46  0  1    4    4
## Mazda RX4 Wag     21.0   6  160 110 3.90 2.875 17.02  0  1    4    4
## Datsun 710        22.8   4  108  93 3.85 2.320 18.61  1  1    4    1
## Hornet 4 Drive    21.4   6  258 110 3.08 3.215 19.44  1  0    3    1
## Hornet Sportabout 18.7   8  360 175 3.15 3.440 17.02  0  0    3    2
## Valiant           18.1   6  225 105 2.76 3.460 20.22  1  0    3    1
```

To gain a deeper understanding of how to apply the pipe, we need to learn the basics of at least one `dplyr` verb. Let's start with `select()` for column selection.

5 Selecting Columns

In chapter 8, you learned that you can select columns with base R brackets by name, like `data[c("col1", "col2")]`. With `dplyr::select()`, you can do the same thing but with a more concise syntax.

The `dplyr` verb `select()` returns the same type of data object, keeps the rows, and changes which columns are shown. It can select columns by name, by ranges such as `x:y`, and by helper patterns.

You can use R operators `:`, `!`, `&` and `|`, `c()`, and `%in%` to select columns. You can also use helper functions like `starts_with()`, `ends_with()`, `contains()`, and `matches()` to select columns based on patterns in their names. For this chapter, we will focus on selecting columns by name and by negative selection with `-`.

The SYNTAX: `select(data, col1, col2, …)`

5.1 Positive Selection

Goal: keep specific columns by name.

```r
#  Type this in a Quarto code chunk
select(mtcars, mpg, cyl, hp)    # keep three columns
```

```
##                    mpg cyl  hp
## Mazda RX4         21.0   6 110
```

```
## Mazda RX4 Wag       21.0   6 110
## Datsun 710          22.8   4  93
## Hornet 4 Drive      21.4   6 110
## Hornet Sportabout   18.7   8 175
## Valiant             18.1   6 105
## Duster 360          14.3   8 245
## Merc 240D           24.4   4  62
## Merc 230            22.8   4  95
## Merc 280            19.2   6 123
## Merc 280C           17.8   6 123
## Merc 450SE          16.4   8 180
## Merc 450SL          17.3   8 180
## Merc 450SLC         15.2   8 180
## Cadillac Fleetwood  10.4   8 205
## Lincoln Continental 10.4   8 215
## Chrysler Imperial   14.7   8 230
## Fiat 128            32.4   4  66
## Honda Civic         30.4   4  52
## Toyota Corolla      33.9   4  65
## Toyota Corona       21.5   4  97
## Dodge Challenger    15.5   8 150
## AMC Javelin         15.2   8 150
## Camaro Z28          13.3   8 245
## Pontiac Firebird    19.2   8 175
## Fiat X1-9           27.3   4  66
## Porsche 914-2       26.0   4  91
## Lotus Europa        30.4   4 113
## Ford Pantera L      15.8   8 264
## Ferrari Dino        19.7   6 175
## Maserati Bora       15.0   8 335
## Volvo 142E          21.4   4 109
```

The `select()` function is used to keep only the `mpg`, `cyl`, and `hp` columns from the `mtcars` dataset. The resulting data frame will contain all rows but only those three columns.

5.2 Negative Selection

In chapter 10 you learned `x[-i, -j]` drops the i-th row and j-th column. In `dplyr::select()`, you can use a leading `-` to drop columns by name.

Goal: remove columns with a leading `-`.

```
#  Type this in a Quarto code chunk
select(mtcars, -gear)   # drop one column
```

The `select()` function is used to drop the `gear` column from the `mtcars` dataset. The resulting data frame will contain all rows and all columns except for `gear`.

Now that you know how to select columns, let's see how to use the pipe to chain operations together.

5.3 Piping with Select

Goal: Let's add a pipe!

```
#  Type this in a Quarto code chunk
mtcars %>%
  select(mpg, cyl, hp)   # easier to read?
```

The pipe `%>%` takes the `mtcars` dataset and feeds it into the `select()` function, which keeps only the `mpg`, `cyl`, and `hp` columns. This version reads more like a sequence of actions: "Start with mtcars, and then select these columns."

Now that you see how to select columns and use the pipe, let's practice storing a subset of a dataset in the environment.

5.4 Subset with Select

We'll mix it up by using a different data set, `gapminder` from `dslabs`. This data set contains information about countries over time.

```
?gapminder
```

Goal: Create a copy of the `gapminder` dataset that contains only specific columns: country, year, infant_mortality, and continent. Name the subset `I_Demo`.

```
#  Type this in a Quarto code chunk:
I_Demo <- gapminder %>%
  select(country, year, infant_mortality, continent)
```

```
#   Verify
head(I_Demo)
```

```
##               country year infant_mortality continent
## 1             Albania 1960           115.40    Europe
## 2             Algeria 1960           148.20    Africa
## 3              Angola 1960           208.00    Africa
## 4 Antigua and Barbuda 1960               NA  Americas
## 5           Argentina 1960            59.87  Americas
## 6             Armenia 1960               NA      Asia
```

A new dataset called `I_Demo` was created from the `gapminder` dataset that only contains the columns `country`, `year`, `infant_mortality`, and `continent`.

Practice: fill in the blank prompt

You try to design your own subsetting operation.

Goal: Write an example that selects only the columns you choose.

Step 1 Identify the column options.

```
#   Pre-check
colnames(gapminder)
```

```
## [1] "country"          "year"             "infant_mortality" "life_expectancy"
## [5] "fertility"        "population"       "gdp"              "continent"
## [9] "region"
```

Step 2 Choose which columns to keep and write the code to create a new dataset with only those columns. Use the exact names of the columns.

```
#  Type this in a Quarto code chunk:
Your_Choice <- gapminder %>%
  select(____, ____, ____)   # choose columns

#   Verify
head(Your_Choice)
```

Now that you know how to store a subset when selecting columns and use the pipe, let's see how to filter rows with `filter()`.

6 Filtering Rows

In chapter 8 and 10, you learned that you can filter rows with base R brackets by name, like `data[data$col1 > 10, ]`. With `dplyr::filter()`, you can do the same thing but with a more concise syntax.

`filter()` keeps only rows for which the condition is `TRUE`. This means that `filter()` treats `NA` values as `FALSE` and drops them. If you want to keep rows with `NA`, you can use the `is.na()` function in your filter condition.

The SYNTAX: `filter(data, condition1, condition2, …)`

You'll often use **logical operators** with `filter()`.

Operator	Meaning	Example
`>`	greater than	mpg **>** 20
`<`	less than	mpg **<** 15
`==`	equal to	am **==** 1
`>=`	greater *or* equal	mpg **>=** 25
`<=`	less *or* equal	mpg **<=** 15
`!`	not	**!**(gear %in% c(3,4))
`!=`	not equal to	gear **!=** 4
`&`	and	x > 2 **&** y == "a"
`|`	or	cyl == 4 **|** cyl == 8
`%in%`	in set	region **%in%** c("West", "South")

Let's start with a simple example of filtering rows with a single condition.

6.1 Single Condition

`filter()` uses logical operators!

```
#  Type this in a Quarto code chunk:
filter(mtcars, mpg >= 25)               # high-mileage cars
```

```
##                 mpg cyl  disp  hp drat    wt  qsec vs am gear carb
## Fiat 128       32.4   4  78.7  66 4.08 2.200 19.47  1  1    4    1
## Honda Civic    30.4   4  75.7  52 4.93 1.615 18.52  1  1    4    2
## Toyota Corolla 33.9   4  71.1  65 4.22 1.835 19.90  1  1    4    1
## Fiat X1-9      27.3   4  79.0  66 4.08 1.935 18.90  1  1    4    1
## Porsche 914-2  26.0   4 120.3  91 4.43 2.140 16.70  0  1    5    2
## Lotus Europa   30.4   4  95.1 113 3.77 1.513 16.90  1  1    5    2
```

6.2 Piping with Filter

We'll mix it up by using a different data set, `murders` from `dslabs`. This data set contains information about gun murders in the US.

```
?murders
```

Prediction: There are 50 states in the US, so there should be 50 rows.

```
#   Pre-check
nrow(murders)
```

```
## [1] 51
```

... Wait, what? ... Question: What are these 51 states?

```
# Test:
unique(murders$state)  # 51 states
```

```
##  [1] "Alabama"              "Alaska"               "Arizona"
##  [4] "Arkansas"             "California"           "Colorado"
##  [7] "Connecticut"          "Delaware"             "District of Columbia"
## [10] "Florida"              "Georgia"              "Hawaii"
## [13] "Idaho"                "Illinois"             "Indiana"
## [16] "Iowa"                 "Kansas"               "Kentucky"
## [19] "Louisiana"            "Maine"                "Maryland"
## [22] "Massachusetts"        "Michigan"             "Minnesota"
## [25] "Mississippi"          "Missouri"             "Montana"
## [28] "Nebraska"             "Nevada"               "New Hampshire"
## [31] "New Jersey"           "New Mexico"           "New York"
## [34] "North Carolina"       "North Dakota"         "Ohio"
## [37] "Oklahoma"             "Oregon"               "Pennsylvania"
## [40] "Rhode Island"         "South Carolina"       "South Dakota"
## [43] "Tennessee"            "Texas"                "Utah"
## [46] "Vermont"              "Virginia"             "Washington"
## [49] "West Virginia"        "Wisconsin"            "Wyoming"
```

Ohhhhh! "District of Columbia" is included in the dataset.

Goal: Create a copy of the `murders` dataset that excludes the District of Columbia.

```
#  Type this in a Quarto code chunk:
No_Washington <- murders %>%
  filter(state != "District of Columbia")
```

```
#   Verify
nrow(No_Washington)  # 50 states
```

A new dataset called `No_Washington` was created from the `murders` dataset that excludes Washington DC.

6.3 Multiple Conditions with AND

We learned in chapter 8 and 10 that we can filter with multiple conditions using `&` for AND. In `dplyr::filter()`, commas (`,`) equal AND (`&`).

6.3.1 And , and &

Let's filter with two conditions! In filter(), multiple conditions separated by commas are combined with &. So these two calls mean the same thing:

```
#  Type this in a Quarto code chunk:
filter(mtcars, mpg >= 25, cyl == 4)
```

```
##                 mpg cyl  disp  hp drat    wt  qsec vs am gear carb
## Fiat 128       32.4   4  78.7  66 4.08 2.200 19.47  1  1    4    1
## Honda Civic    30.4   4  75.7  52 4.93 1.615 18.52  1  1    4    2
## Toyota Corolla 33.9   4  71.1  65 4.22 1.835 19.90  1  1    4    1
## Fiat X1-9      27.3   4  79.0  66 4.08 1.935 18.90  1  1    4    1
## Porsche 914-2  26.0   4 120.3  91 4.43 2.140 16.70  0  1    5    2
## Lotus Europa   30.4   4  95.1 113 3.77 1.513 16.90  1  1    5    2
```

```
#  Type this in a Quarto code chunk:
filter(mtcars, mpg >= 25 & cyl == 4)
```

```
##                 mpg cyl  disp  hp drat    wt  qsec vs am gear carb
## Fiat 128       32.4   4  78.7  66 4.08 2.200 19.47  1  1    4    1
## Honda Civic    30.4   4  75.7  52 4.93 1.615 18.52  1  1    4    2
## Toyota Corolla 33.9   4  71.1  65 4.22 1.835 19.90  1  1    4    1
## Fiat X1-9      27.3   4  79.0  66 4.08 1.935 18.90  1  1    4    1
## Porsche 914-2  26.0   4 120.3  91 4.43 2.140 16.70  0  1    5    2
## Lotus Europa   30.4   4  95.1 113 3.77 1.513 16.90  1  1    5    2
```

Both of these code chunks will return the same result: a subset of the `mtcars` dataset that includes only rows where `mpg` is greater than or equal to 25 and `cyl` is equal to 4. The first chunk uses commas to separate conditions, while the second chunk uses the `&` operator to combine them. Both are valid ways to specify multiple conditions in `filter()`.

The catch is that comma is only a shortcut inside `filter()` for separating multiple conditions. It is not a general replacement for `&` in R, and it does not help when you need more complex grouping with `|`.

6.3.2 With the pipe

Goal: Let's filter with two conditions with a pipe and store it as `efficient`

Question: How many cars have *both* mpg 25 *and* 4 cylinders?

```
#  Type this in a Quarto code chunk:
efficient <- mtcars %>%
  filter(mpg >= 25, cyl == 4)           # two conditions
```

```
#   Verify
nrow(efficient)  # how many cars are efficient?
```

```
## [1] 6
```

A new dataset called `efficient` was created from the `mtcars` dataset that includes only rows where `mpg` is greater than or equal to 25 *and* `cyl` is equal to 4. The number of rows in the `efficient` dataset indicates how many cars in the dataset meet **both** conditions (n = 6).

6.4 Multiple Conditions with OR

In `dplyr::filter()`, multiple conditions separated by commas are combined with AND, not OR. So when you want OR logic, you must write `|` explicitly inside the condition. In base R, `|` is the elementwise OR operator, and `%in%` returns `TRUE` when a value appears in a set of values.

6.4.1 Or | and %in%

Goal: Let's filter with OR! When you want rows that match either one condition or another, use `|`.

```r
#  Type this in a Quarto code chunk:
filter(mtcars, cyl == 4 | cyl == 6)
```

```
##                 mpg cyl  disp  hp drat    wt  qsec vs am gear carb
## Mazda RX4      21.0   6 160.0 110 3.90 2.620 16.46  0  1    4    4
## Mazda RX4 Wag  21.0   6 160.0 110 3.90 2.875 17.02  0  1    4    4
## Datsun 710     22.8   4 108.0  93 3.85 2.320 18.61  1  1    4    1
## Hornet 4 Drive 21.4   6 258.0 110 3.08 3.215 19.44  1  0    3    1
## Valiant        18.1   6 225.0 105 2.76 3.460 20.22  1  0    3    1
## Merc 240D      24.4   4 146.7  62 3.69 3.190 20.00  1  0    4    2
## Merc 230       22.8   4 140.8  95 3.92 3.150 22.90  1  0    4    2
## Merc 280       19.2   6 167.6 123 3.92 3.440 18.30  1  0    4    4
## Merc 280C      17.8   6 167.6 123 3.92 3.440 18.90  1  0    4    4
## Fiat 128       32.4   4  78.7  66 4.08 2.200 19.47  1  1    4    1
## Honda Civic    30.4   4  75.7  52 4.93 1.615 18.52  1  1    4    2
## Toyota Corolla 33.9   4  71.1  65 4.22 1.835 19.90  1  1    4    1
## Toyota Corona  21.5   4 120.1  97 3.70 2.465 20.01  1  0    3    1
## Fiat X1-9      27.3   4  79.0  66 4.08 1.935 18.90  1  1    4    1
## Porsche 914-2  26.0   4 120.3  91 4.43 2.140 16.70  0  1    5    2
## Lotus Europa   30.4   4  95.1 113 3.77 1.513 16.90  1  1    5    2
## Ferrari Dino   19.7   6 145.0 175 3.62 2.770 15.50  0  1    5    6
## Volvo 142E     21.4   4 121.0 109 4.11 2.780 18.60  1  1    4    2
```

```r
#  Type this in a Quarto code chunk:
filter(mtcars, cyl %in% c(4, 6))
```

```
##                 mpg cyl  disp  hp drat    wt  qsec vs am gear carb
## Mazda RX4      21.0   6 160.0 110 3.90 2.620 16.46  0  1    4    4
## Mazda RX4 Wag  21.0   6 160.0 110 3.90 2.875 17.02  0  1    4    4
## Datsun 710     22.8   4 108.0  93 3.85 2.320 18.61  1  1    4    1
## Hornet 4 Drive 21.4   6 258.0 110 3.08 3.215 19.44  1  0    3    1
## Valiant        18.1   6 225.0 105 2.76 3.460 20.22  1  0    3    1
## Merc 240D      24.4   4 146.7  62 3.69 3.190 20.00  1  0    4    2
## Merc 230       22.8   4 140.8  95 3.92 3.150 22.90  1  0    4    2
## Merc 280       19.2   6 167.6 123 3.92 3.440 18.30  1  0    4    4
## Merc 280C      17.8   6 167.6 123 3.92 3.440 18.90  1  0    4    4
## Fiat 128       32.4   4  78.7  66 4.08 2.200 19.47  1  1    4    1
## Honda Civic    30.4   4  75.7  52 4.93 1.615 18.52  1  1    4    2
## Toyota Corolla 33.9   4  71.1  65 4.22 1.835 19.90  1  1    4    1
## Toyota Corona  21.5   4 120.1  97 3.70 2.465 20.01  1  0    3    1
## Fiat X1-9      27.3   4  79.0  66 4.08 1.935 18.90  1  1    4    1
## Porsche 914-2  26.0   4 120.3  91 4.43 2.140 16.70  0  1    5    2
## Lotus Europa   30.4   4  95.1 113 3.77 1.513 16.90  1  1    5    2
## Ferrari Dino   19.7   6 145.0 175 3.62 2.770 15.50  0  1    5    6
## Volvo 142E     21.4   4 121.0 109 4.11 2.780 18.60  1  1    4    2
```

Both of these code chunks return the same result: a subset of the `mtcars` dataset that includes only rows where `cyl` is equal to 4 or 6. The first chunk uses the `|` operator to say "4 cylinders or 6 cylinders." The second chunk uses `%in%` to check whether `cyl` is one of the values in `c(4, 6)`. Both are valid, but %in% is often easier to read when you are checking one variable against several possible values.

The catch is that `%in%` is only a shortcut when you are comparing one variable to multiple values. It is not a general replacement for `|`. For example, if you want cars with 4 cylinders or cars with `mpg >= 30`, you must use `|` because those are two different conditions on different variables.

Goal: Let's filter with OR on different variables.

```
#  Type this in a Quarto code chunk:
filter(mtcars, cyl == 4 | mpg >= 30)  # different variables
```

```
##                 mpg cyl  disp  hp drat    wt  qsec vs am gear carb
## Datsun 710     22.8   4 108.0  93 3.85 2.320 18.61  1  1    4    1
## Merc 240D      24.4   4 146.7  62 3.69 3.190 20.00  1  0    4    2
## Merc 230       22.8   4 140.8  95 3.92 3.150 22.90  1  0    4    2
## Fiat 128       32.4   4  78.7  66 4.08 2.200 19.47  1  1    4    1
## Honda Civic    30.4   4  75.7  52 4.93 1.615 18.52  1  1    4    2
## Toyota Corolla 33.9   4  71.1  65 4.22 1.835 19.90  1  1    4    1
## Toyota Corona  21.5   4 120.1  97 3.70 2.465 20.01  1  0    3    1
## Fiat X1-9      27.3   4  79.0  66 4.08 1.935 18.90  1  1    4    1
## Porsche 914-2  26.0   4 120.3  91 4.43 2.140 16.70  0  1    5    2
## Lotus Europa   30.4   4  95.1 113 3.77 1.513 16.90  1  1    5    2
## Volvo 142E     21.4   4 121.0 109 4.11 2.780 18.60  1  1    4    2
```

This code chunk returns a subset of the `mtcars` dataset that includes rows where either `cyl` is equal to 4 or `mpg` is greater than or equal to 30. Since these conditions involve different variables, you cannot use `%in%` and must use the `|` operator to combine them.

6.4.2 With the pipe

Goal: Let's filter with two conditions using OR (|), piping, and storing the subset to the environment.

Question: How many cars have *either* mpg 25 *or* 4 cylinders?

```
#  Type this in a Quarto code chunk:
efficient_ish <- mtcars %>%
  filter(mpg >= 25 | cyl == 4)    # two conditions
```

```
#   Verify
nrow(efficient_ish)  # how many cars are efficient_ish?
```

```
## [1] 11
```

A new dataset called `efficient_ish` was created from the `mtcars` dataset that includes rows where either `mpg` is greater than or equal to 25 or `cyl` is equal to 4. The number of rows in the `efficient_ish` dataset indicates how many cars meet *at least one* of those conditions. (n = 11)

7 filter() AND select()

So far, we've used `select()` to changes which columns we keep and `filter()` to change which rows we keep. Now let's combine them together.

Review

- `filter()` looks at the values in a data frame and keeps only rows where the condition is `TRUE`.
- `select()` looks at column names and keeps only the variables you ask for.

Combining them lets you subset a data frame by both rows and columns.

The SYNTAX:

`data %>% filter(condition) %>% select(col1, col2, …)`

A safe beginner workflow habit is to `filter()` first and `select()` second. That way, all columns are still available when you write your filtering conditions. If you `select()` first and accidentally remove a column needed by `filter()`, your code will fail.

We'll mix it up by using a different data set, `us_contagious_diseases` from `dslabs`. This data set contains information about contagious diseases in the US.

```
?us_contagious_diseases
```

Goal: Using `us_contagious_diseases` from `dslabs`, keep only rows for `Alaska` from `1980` and later, then keep only the columns `year`, `disease`, and `count`. Store as `alaska_disease`.

```
#  Type this in a Quarto code chunk:
alaska_disease <- us_contagious_diseases %>%
  filter(state == "Alaska", year >= 1980) %>%   # rows
  select(year, disease, count)                  # columns
```

This pipeline says: start with `us_contagious_diseases`, keep only the rows for Alaska in 1980 or later, and then trim the result down to just three columns. `filter()` changes the number of rows. `select()` changes the number of columns. That is the key difference between the two verbs.

```
#   Verify
head(alaska_disease)
```

```
##   year     disease count
## 1 1980 Hepatitis A    18
## 2 1981 Hepatitis A    45
## 3 1982 Hepatitis A    17
## 4 1983 Hepatitis A    43
## 5 1984 Hepatitis A    14
## 6 1985 Hepatitis A    18
```

A new dataset called `alaska_disease` was created from `us_contagious_diseases`. First, `filter()` kept only rows where `state` is `"Alaska"` and year is greater than or equal to `1980`. Then, `select()` kept only the columns `year`, `disease`, and `count`. Notice that state was used in the filtering step, but it does not appear in the final result because `select()` removed it afterward.

8 One Application

"Which automatic cars (`am == 0`) in `mtcars` have horsepower > 150?

Goal: Answer the question. Show only `hp` and `mpg`.

In `mtcars`, `am` is coded as `0 = automatic` and `1 = manual` horsepower is `hp` and miles per gallon is `mpg`.

```
# Test
powerful_auto <- mtcars %>%
  filter(am == 0, hp > 150) %>%
  select(hp, mpg)
```

```
#   Verify
head(powerful_auto)
```

```
##                     hp  mpg
## Hornet Sportabout  175 18.7
## Duster 360         245 14.3
## Merc 450SE         180 16.4
## Merc 450SL         180 17.3
## Merc 450SLC        180 15.2
## Cadillac Fleetwood 205 10.4
```

```
nrow(powerful_auto)
```

```
## [1] 10
```

A new dataset called `powerful_auto` was created from `mtcars`. The `filter()` step kept only cars that are automatic (`am == 0`) and have more than 150 horsepower. The `select()` step then kept only the `hp` and `mpg` columns, because those are the variables needed for the report. The result contains 10 cars.

9 Summary

In Chapter 11, you learned how to use `dplyr` to create smaller, analysis-ready datasets in a clear and reproducible way. You practiced `select()` to keep or drop columns, `filter()` to keep rows based on logical conditions, and `%>%` to connect multiple steps into a readable workflow. You also worked in Quarto so that your code, output, and written notes stay together in one document.

You applied comparison operators and Boolean logic to ask more precise questions of data, including single-condition filters, multiple-condition filters with AND/OR logic, and workflows that combine `filter()` and `select()` in the same pipeline. The chapter also extended subsetting into practical tasks such as, previewing results, and creating focused subsets from real datasets for exploration and reporting.

10 Chapter Terms

Logical Operators: Operators that create or combine logical elements. The most common are comparison operators—`==`, `!=`, `<`, `>`, `<=`, `>=`—which return `TRUE`/`FALSE`, and Boolean operators—`!` (NOT), `&` / `|` (AND/OR applied element-by-element), and `&&` / `||` (AND/OR that evaluate only the first element, mainly for if conditions).

Pipe: An operator that passes the result of one step directly into the next step, letting you write code as a readable sequence of actions instead of nesting functions inside each other. In tidyverse code, the pipe is usually `%>%`, so `x %>% f()` means "take `x`, then apply `f()`"; in newer base R, a similar built-in pipe is `|>`.

Tidyverse: An opinionated collection of R packages designed to work together for data science. The packages share a common design philosophy, data structures, and coding style, which makes it easier to import, clean, reshape, visualize, and analyze data within a consistent workflow.

Quarto: An open-source scientific and technical publishing system that uses Quarto Markdown files (`.qmd`) to combine Markdown text with executable code (including R) and render reproducible outputs like HTML, PDF, Word, slides, and websites. It's the language-agnostic successor to R Markdown, designed for reports where code, results, and writing stay tied together.

YAML Header: The metadata section at the top of an R Markdown or Quarto document, written between `---` lines, that controls document settings like the title, author, date, output format (HTML/PDF/Word), and options for rendering.

11 Practice Space

In this Practice Space, you will practice using a Quarto document as a reproducible workspace while learning two core `dplyr` verbs:

- `select()` to keep specific columns
- `filter()` to keep specific rows

You will also practice basic Quarto habits from this chapter:

- saving your work as a `.qmd` file
- writing memo responses as normal text outside code chunks
- using Quarto chunk controls with `#|` as you choose to show or hide code, messages, and warnings

For this Practice Space, create a new Quarto document and save it in your course folder as:

`chapter11_practice.qmd`

Use headings as you work so your file is organized and readable.

You won't "render" this file. For the file submission assignment, you will submit the `.qmd` file.

Suggested Headings

```
# Chapter 11 Practice

## Task 0

## Task 1

## Task 2

## Task 3

## Task 4

## Task 5

## Task 6

## Task 7
```

11.1 Task 0

Load the packages needed for this Practice Space: `dplyr` and `dslabs`. This setup chunk is also as great place to practice Quarto chunk controls like `#| message: false` and `#| warning: false`

Write the goal of this task in your own words.

```
#  Type this in Quarto code chunk, fill in the blanks
_______(dslabs)
_______(dplyr)
```

11.2 Task 1

Column Selection

Create `C_Overload` that keeps only `year` and `carbon_emissions` from the `temp_carbon` data frame. Use `select()` without `%>%`.

Write the goal of this task in your own words.

```
#  Type this in Quarto code chunk, fill in the blanks
C_Overload <- _____
```

```
#   Verify
head(C_Overload)
```

Explain what `select()` did in this task. How does head help you check that it worked?

11.3 Task 2

Single-Column Dataframe

Pipe the `stars` dataset in `dslabs` through `select()` to keep only the `star` column. Name the result `star_names`.

Write the goal of this task in your own words.

```
#  Type this in Quarto code chunk, fill in the blanks
star_names <- _____
```

```
#   Verify
head(star_names)
```

Does this return a vector or keep a one-column data frame?

11.4 Task 3

Row Filtering

From `murders`, remove every row where state equals `"Florida"` using `!=`. Name the object `No_Florida`.

Write the goal of this task in your own words.

```
#   Pre-test
unique(murders$state)
```

```
#  Type this in Quarto code chunk, fill in the blanks
No_Florida <- _____
```

```
#   Verify
unique(No_Florida$state)
```

Why might pre-checking the unique values of `state` in the data frame `murders` useful? What change are you expecting to see in the Verify step? Explain what rows are removed.

11.5 Task 4

Multiple Conditions

From `mtcars`, using `filter()` and piping, pull cars in `mtcars` that are **manual**, have `8` **cylinders**, and have **1/4 mile time** *less than* `15`. Store as `Fast_n_Furious`.

Hint: use `?mtcars` to identify the relevant variables for each condition.

Write the goal of this task in your own words.

```
summary(mtcars$qsec)  # check qsec range
```

```
#  Type this in Quarto code chunk, fill in the blanks
Fast_n_Furious <- _____
```

```
#   Verify
head(Fast_n_Furious)
nrow(Fast_n_Furious)
```

Explain the conditions you used to filter the rows. What does each condition mean in terms of the characteristics of the cars? How many cars meet all three conditions?

11.6 Task 5

OR Logic + Column Slice

From `mtcars`, keep only the cars whose displacement is either **under 160** or **over 250**, and then show only the first three columns of those rows.(Hint: `:`). Store as `filtered_cars`

Write the goal of this task in your own words.

```
#   Pre-check
range(mtcars$disp)  # check disp
```

This tells the spread of `disp` in `mtcars` so you can see whether the cutoffs `160` and `250` make sense.

```
#  Type this in Quarto code chunk, fill in the blanks
filtered_cars <- _____
```

```
#   Verify
all(filtered_cars$disp > 250 | filtered_cars$disp < 160) # check disp
```

The function `all()` to test whether every kept row satisfies the condition

11.7 Task 6

Reflection

Write a few sentences comparing the utility of `dplyr` verbs `select()` and `filter()` as it compares to base-R brackets `[ ]` for subsetting.

11.8 Task 7

Self-check checklist

For the file submission, make sure your qmd creates runs from top to bottom without errors and that the following objects are created in your environment with the expected content:

- `C_Overload` exists and has only `year` and `carbon_emissions`.
- `star_names` exists and has one column named `star`.
- `No_Florida` has 0 rows with `state == 'Florida'`.
- `Fast_n_Furious` with rows that only contain manual, 8 cylinders, and where `qsec < 15`.
- `filtered_cars` with rows where `disp > 250` OR `disp < 160` and only the first three columns.
- **All code in your Quarto document runs top-to-bottom without errors.**

Goal: Check `.qmd` for submission quality.

Step 1: Sweep your Environment (start clean).
Click the **broom icon** in the Environment pane to clear objects.

Step 2: Run everything top-to-bottom.
Use **Run → Run All** to execute every chunk in order.

Step 3: Save and submit.
After your notebook runs without errors, make sure the latest edits are save your file (`.qmd`)